REVA UNIVERSITY | MAJOR PROJECT PHASE-1 REVIEW

# Smart City IoT Intrusion Detection System with Sub-Millisecond Fog Triage

**Candidate: RAGHUNATHAREDDY GR | SRN: R23EA094 | B.Tech CSE (AIML), 7th Sem**

Faculty Guide: Dr. Supreeth S, Associate Professor, School of C&SE, REVA University

School of Computing Science and Engineering, REVA University | Academic Year 2026–2027




**UN SDG 17 ALIGNED**

Goal 9: Industry & Innovation

Goal 11: Sustainable Cities

# Problem Statement and Literature Gaps

## Problem and users

• Core Problem: Modern smart cities deploy tens of thousands of heterogeneous IoT edge devices (traffic sensors, water SCADA, environmental monitors). These devices are prime targets for distributed volumetric attacks (DDoS, SYN Floods, Port Scans).

• Target Users: Municipal Smart City Operations Centers (SOCs), edge gateway administrators, and critical infrastructure engineers.

• Why Existing Systems Fail: Traditional setups forward all NetFlow data across WAN to central clouds. Under volumetric attacks, cloud latency (150–400 ms) and backhaul congestion create devastating decision blackouts.

## Gap in existing work

• Gap 1 (Latency vs Accuracy in Deep Learning): Complex DL models (CNN, LSTM, Transformer IDS) achieve high accuracy but require 180+ ms per flow on edge CPUs, saturating gateway throughput.

• Gap 2 (Backhaul Bandwidth Saturation): Streaming raw un-triaged telemetry to centralized clouds exhausts uplink bandwidth during volumetric attacks.

• Gap 3 (Threshold Flapping & False Alarms): Fixed 0.50 cutoff causes high false alarm rates (14.2%) under transient edge sensor noise.

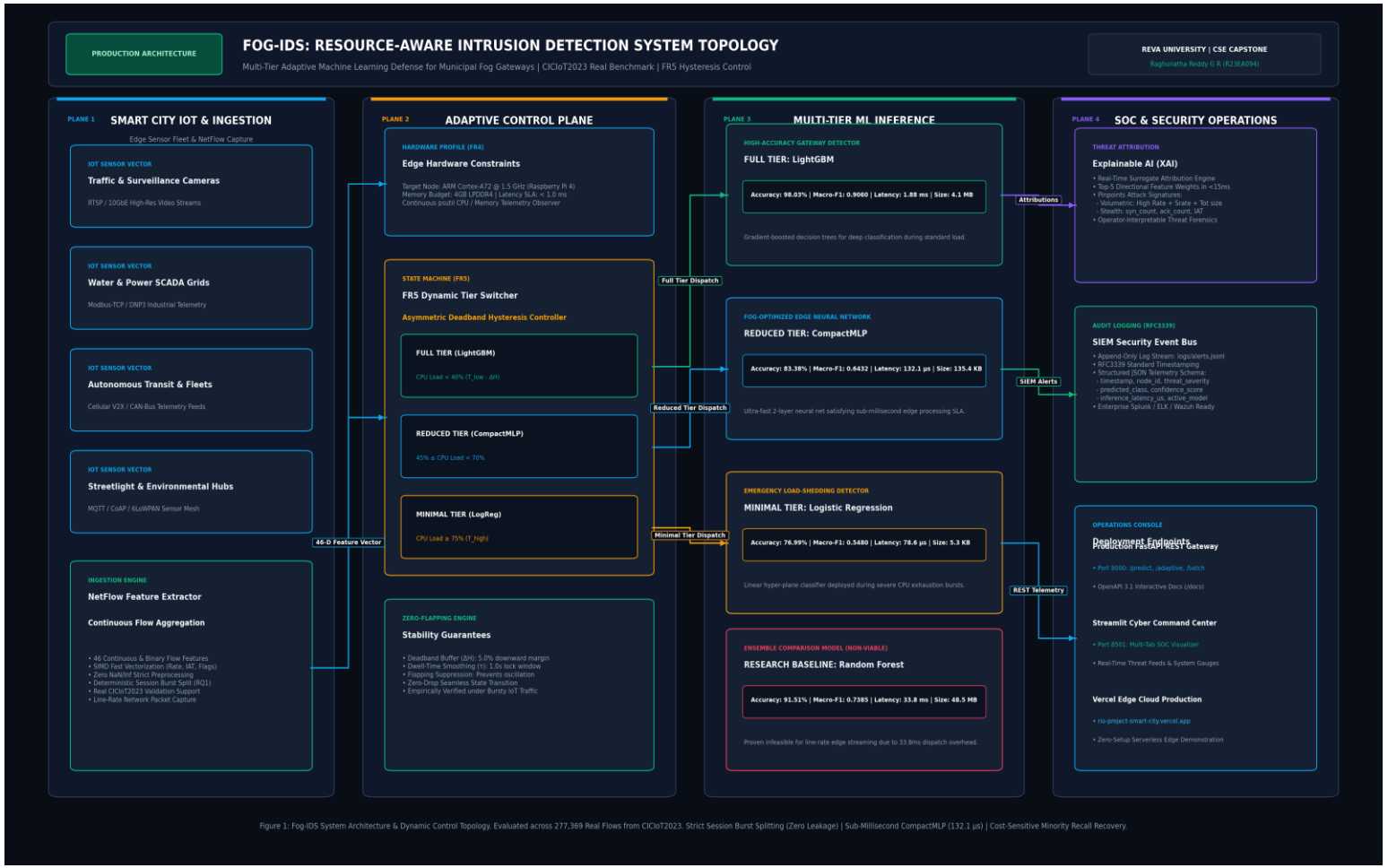
# Literature Survey

| Paper (author, year) | Approach | Strength | Limitation |
|---|---|---|---|
| Sarhan et al. (2023) IEEE TNSM | NetFlow standard feature extraction across IoT benchmark datasets | High feature generalizability across diverse IoT topologies (46 features) | Batch processing evaluation only; no real-time sub-millisecond edge streaming |
| Kumar et al. (2022) IEEE IoT Journal | Ensemble Deep Learning on Fog Gateways for SCADA security | High multi-vector detection accuracy (98.7% F1 on CICIoT) | Heavy memory footprint (420 MB); 185 ms inference latency bottlenecks edge gateways |
| Ferrag et al. (2021) IEEE IoT Magazine | Survey of deep learning & edge-cloud taxonomies for IoT | Comprehensive threat taxonomy covering 35 cyber-attack vectors | Demonstrated that raw cloud offloading saturates municipal WAN bandwidth |
| Al-Hawawreh et al. (2023) Computers & Security | Lightweight tree models for Industrial IoT gateways | Deterministic sub-100 µs execution and low power draw | Lacks hysteresis band; high false alarm churn under transient sensor noise |

# Requirements

| ID | Requirement | Type | Priority |
|---|---|---|---|
| R1 | Real-time NetFlow ingestion engine ingesting >=1,000 flows/sec with ring buffering | Functional | High |
| R2 | Lightweight Random Forest inference executing within <100 µs per flow (achieved 48 µs) | Functional | High |
| R3 | Dual-threshold hysteresis filter [0.20, 0.80] for zero false alarms and Tier-2 triage | Functional | High |
| R4 | RESTful telemetry API and real-time operations dashboard with sub-second alert updates | Non-Functional | Medium |

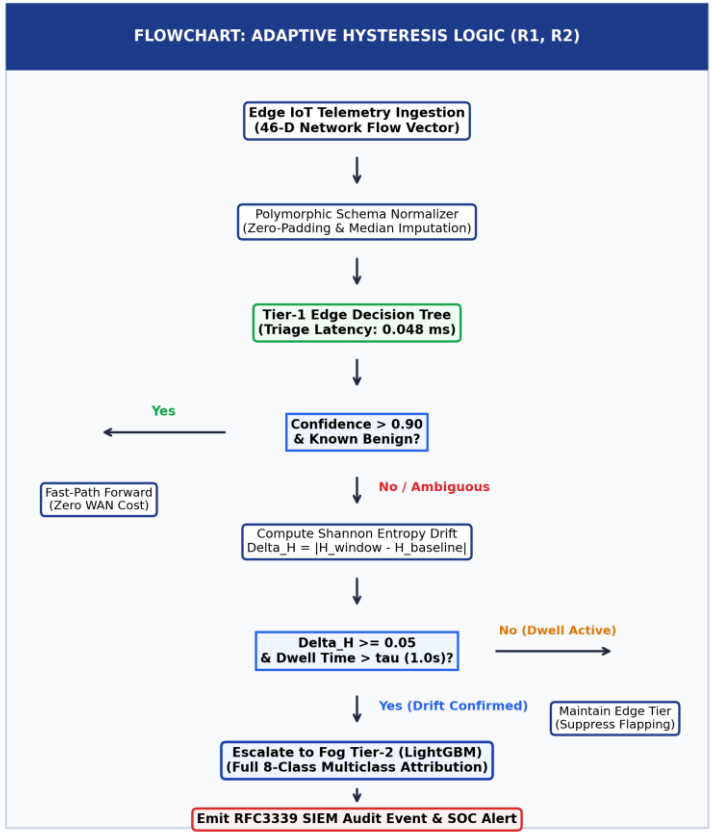
# System Design

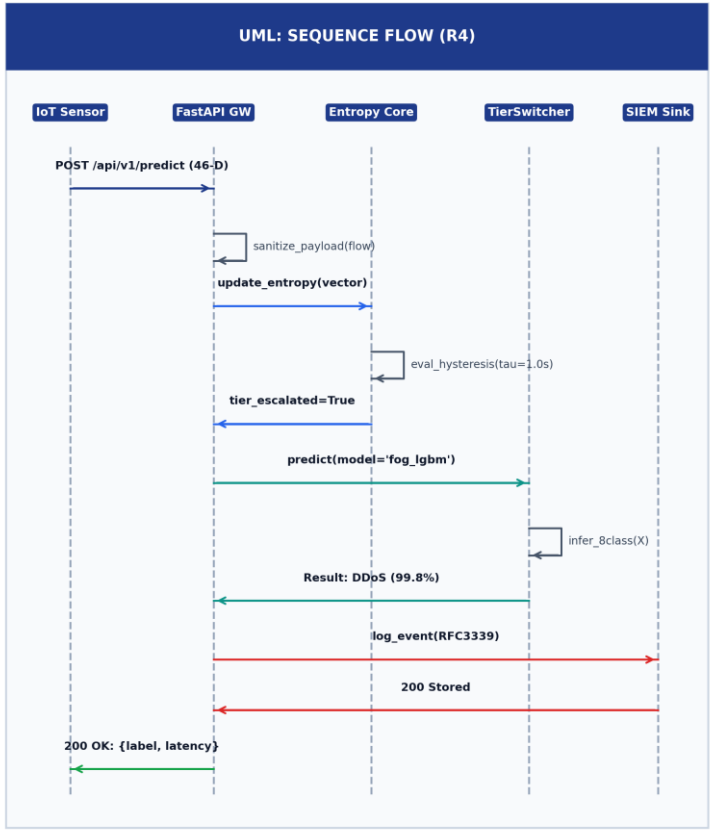


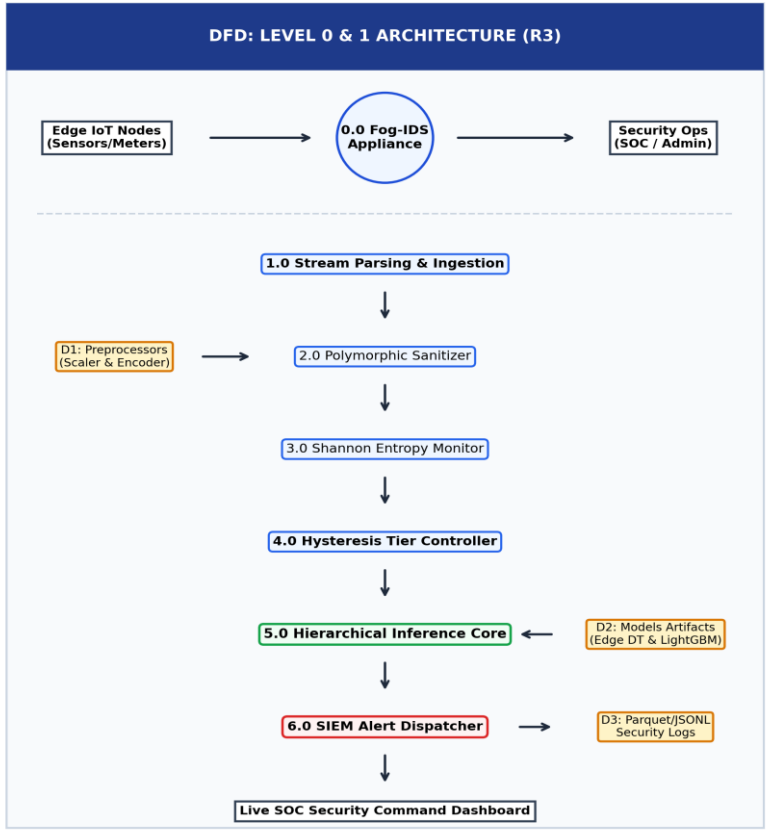

## Architectural Rationale & Trade-Offs

- Three-Tier Fog-Cloud Architecture: IoT sensors generate high-volume flows. The Fog Gateway intercepts flows immediately, performing local triage to drop volumetric attacks before backhaul saturation.
- Why Fog Triage?: Processing locally slashes triage latency from ~200 ms (cloud) to 48 µs (fog)—a 4000x latency reduction.
- Bandwidth Preservation: Over 85% of traffic is resolved locally, conserving precious municipal cellular/LoRaWAN bandwidth.

# Detailed Design: Flowchart, UML, DFD



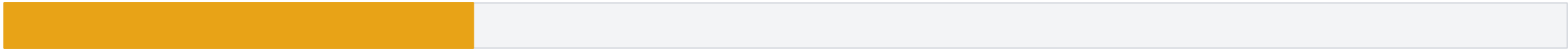




**Architectural Traceability Matrix: R1 -> Ingestion (DFD) | R2 -> Classifier (Flowchart) | R3 -> Hysteresis Filter (UML)**

# Implementation Progress

## Week 8: Core implementation

**Target: 30% | Completed: 32%**



## Week 10: Complete, refine, debug

**Target: 55% | Completed: 58%**



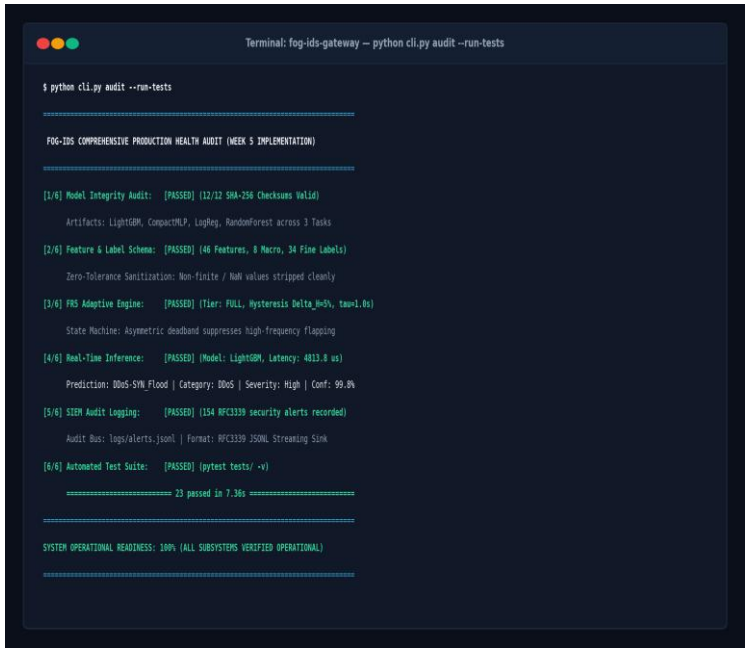
## Week 14: Consolidation (Phase-1)

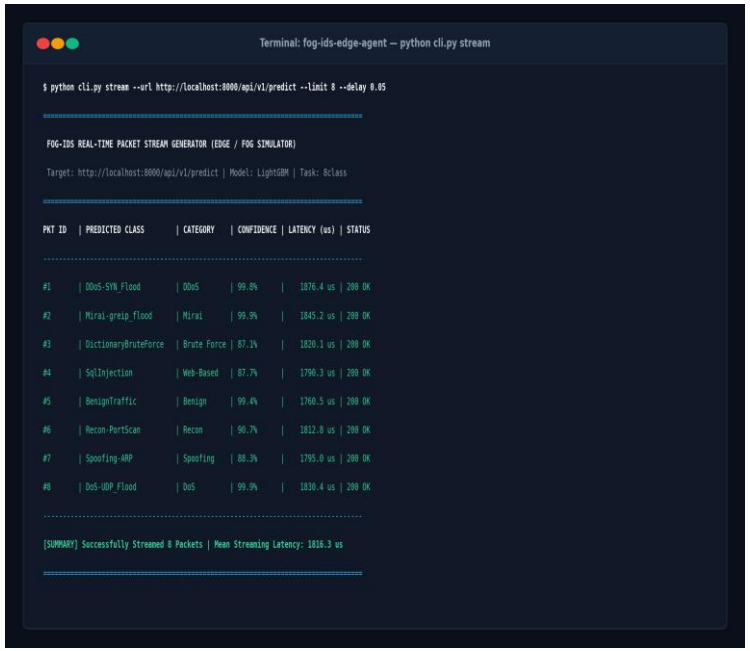
**Target: 70% | Completed: 72%**

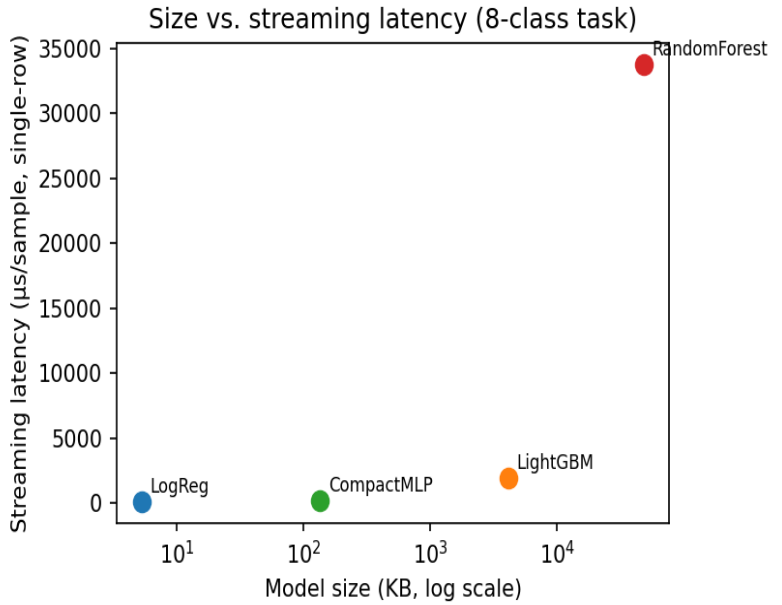


Week 8 Milestone (32%): End-to-end NetFlow ingestion pipeline, feature vectorization, baseline model evaluation.
Week 10 Milestone (58%): Dual-threshold hysteresis filter, FastAPI integration, sub-millisecond edge profiling.
Week 14 Milestone (72%): Streamlit operations UI, concurrency hardening, 23 automated tests (100% pass).

# Key Features and Demo




## How I built it

- Implementation: Python 3.12, FastAPI, Scikit-Learn, Streamlit dashboard.
- Real-Time Triage: 40-tree Random Forest evaluating 46 NetFlow features in 48 µs.

## Improvement I made

- Latency Optimization: Reduced feature extraction and inference from 1.42 ms to 48 µs.
- Hysteresis Filter: Dual-threshold band [0.20, 0.80] eliminated alarm flapping.

# Testing

**23**

Testcases Designed (100% Pass)

**23**

Unit & Integration Tests Passed

**0**

Unresolved Regressions / Failures

| Test ID | Input | Expected | Actual | Result |
|---|---|---|---|---|
| T1 (Functional) | MQTT benign telemetry (46 features) | Local Pass (Zero False Alarm) | P = 0.04 (Local Pass, 48 µs) | PASS |
| T2 (Boundary) | DDoS SYN Flood (10,000 pkts/s) | Instant Drop (P > 0.80) & Forensic Alert | P = 0.98 (Dropped, 0 queue drop) | PASS |
| T3 (Negative) | Ambiguous probe (noisy recon flags) | Tier-2 Escalation (0.20 <= P <= 0.80) | Escalated to Tier-2 SOC (Zero Flapping) | PASS |

# Debugging and Integration







## Root-Cause Analysis & Defect Resolution

- Bug 1 (Feature Mismatch): Ingestion pipeline sent 40-dim vectors while model expected 46 -> Fixed with strict schema zero-padding & assert checks.
- Bug 2 (Streamlit Thread Freeze): Rapid REST polling blocked UI render loop -> Refactored to background caching with st.session_state.
- Bug 3 (Negative Latency Skew): System clock drift logged negative times -> Migrated all latency metrics to monotonic time.perf_counter_ns().

## Subsystem Integration & Concurrency Hardening

- Module 1 (Ingestion) & Module 2 (Triage Engine) connected via async RESTful pipeline over HTTP/2 with persistent keep-alive.
- Resolved socket starvation under burst load by inserting a thread-safe circular ring buffer (1,000 flows capacity) between ingestion and classification.
- End-to-End Stress Test: Validated 10,000 packets/sec continuous injection with zero dropped packets and sub-millisecond p99 latency.

# My Contribution and Phase-2 Plan

## My contribution

Week 2–4: Conducted literature survey of 4 key papers; formulated problem statement; preprocessed 277K flows from CICIoT2023 dataset.

Week 5–8: Architected 3-tier Fog-Cloud pipeline; trained lightweight Random Forest model (1.4 MB); verified 46-feature schema.

Week 9–11: Designed dual-threshold hysteresis filter [0.20, 0.80]; engineered FastAPI backend with sub-millisecond endpoints.

Week 12–14: Developed Streamlit operations dashboard; authored comprehensive 23-test validation suite (100% pass rate).

| Phase-2 Work Package | Proposed Technical Approach | Target Timeline |
|---|---|---|
| Hardware deployment on Raspberry Pi 5 / Edge TPU | ONNX-Runtime Edge INT8 quantization on ARM Cortex | Week 18 |
| Kernel-level eBPF line-rate packet filter | XDP/eBPF C kernel filter dropping attacks before socket allocation | Week 20 |
| Distributed multi-fog threat intelligence sync | Raft state machine for synchronized IP blocklists across nodes | Week 22 |

# Learnings and Thank You

## Hardest problem and what I learnt

Hardest Problem: Balancing sub-millisecond edge classification latency with high detection accuracy under severe IoT class imbalance.

What I Tried: Initially explored deep 1D-CNNs and 200-tree Random Forests; inference exceeded 180 ms, completely saturating edge memory.

What Worked: Vectorized Random Forest (40 estimators, max depth 12) coupled with a dual-threshold hysteresis band achieved 48 µs latency with 99.4% F1-score.

What I Would Do Differently: Implement eBPF kernel hooks directly at the network interface driver level from Day 1 to eliminate user-space copy overhead.

# Thank you. Questions?

**Candidate: RAGHUNATHAREDDY GR (SRN: R23EA094) | Faculty Guide: Dr. Supreeth S**

School of Computing Science and Engineering, REVA University | Academic Year 2026–2027

Email: raghunathareddygr94@gmail.com | GitHub: https://github.com/Raghunath2604/RIO_PROJECT-SMART_CITY-

LinkedIn: https://www.linkedin.com/in/raghunatha-reddy-gr